Properties of cryptographic hash functions

One-way hash function - OWHF

Ease of computation: it is easy to compute $h(x)$

Compression: the hash function maps inputs of arbitrary length to fixed length results

Pre-image resistance (one-way): given y, it is computationally infeasible to find x so that $h(x)=y$

2nd pre-image resistance: given x, it is computationally infeasible to find x', $x \neq x'$, so that $h(x)=h(x')$

Collision resistance: it is computationally infeasible to find x and x', $x \neq x'$, so that $h(x)=h(x')$

Collision-resistant hash function - CRHF

Checksums

The result of applying a hash function is called hash value, message digest, or checksum.

The last term creates frequent confusion:

- In communications, checksums often refer to error correcting codes, typically a cyclic redundancy check (CRC).
- Checksums used by anti-virus products, on the other hand, must not be computed with a CRC but with a cryptographic hash function.

Note: A cyclic redundancy check (CRC) is not a cryptographic hash function: CRC's are designed for detecting transmission errors

Some use cases of crytographic hashes

- Verification of data integrity, signatures
- File identification (P2P, git, ...)

CRHF

- Password storage and verification
- Key derivation

OWHF

- Message authentication codes
- Pseudorandom generation

see later

Birthday paradox

It depends on the bit-length of the hash how probable it is to find collisions by accident.

- Given an *n*-bit hash *y*, the expected number of tries before an *x* with *h(x)=y* is found is 2^n.
- Given *n*-bit hash values, a set of $2^{n/2}$ inputs is likely to contain a pair causing a collision.
- **Birthday paradox**: put *m* balls numbered *1* to *m* into an urn; draw a ball, list its number, and put it back; repeat; for $m \to \infty$, the expected number of draws before a previously drawn number appears is

$$\sqrt{0.5 \cdot \pi \cdot m}$$

Complexity of brute-force attacks

Output size of hash function: n bit

Pre-image and 2nd pre-image:

- Average complexity: $O(2^n)$

Collision: birthday „attack“:

- Average complexity: $O(2^{0.5n})$

Illustration of complexity:

1 Gops/s, 100 parallel ops/unit, 1000 units:

2^{64} ops in about 50 hours

2^{80} ops in about 380 years

2^{128} ops in about 10^{17} years

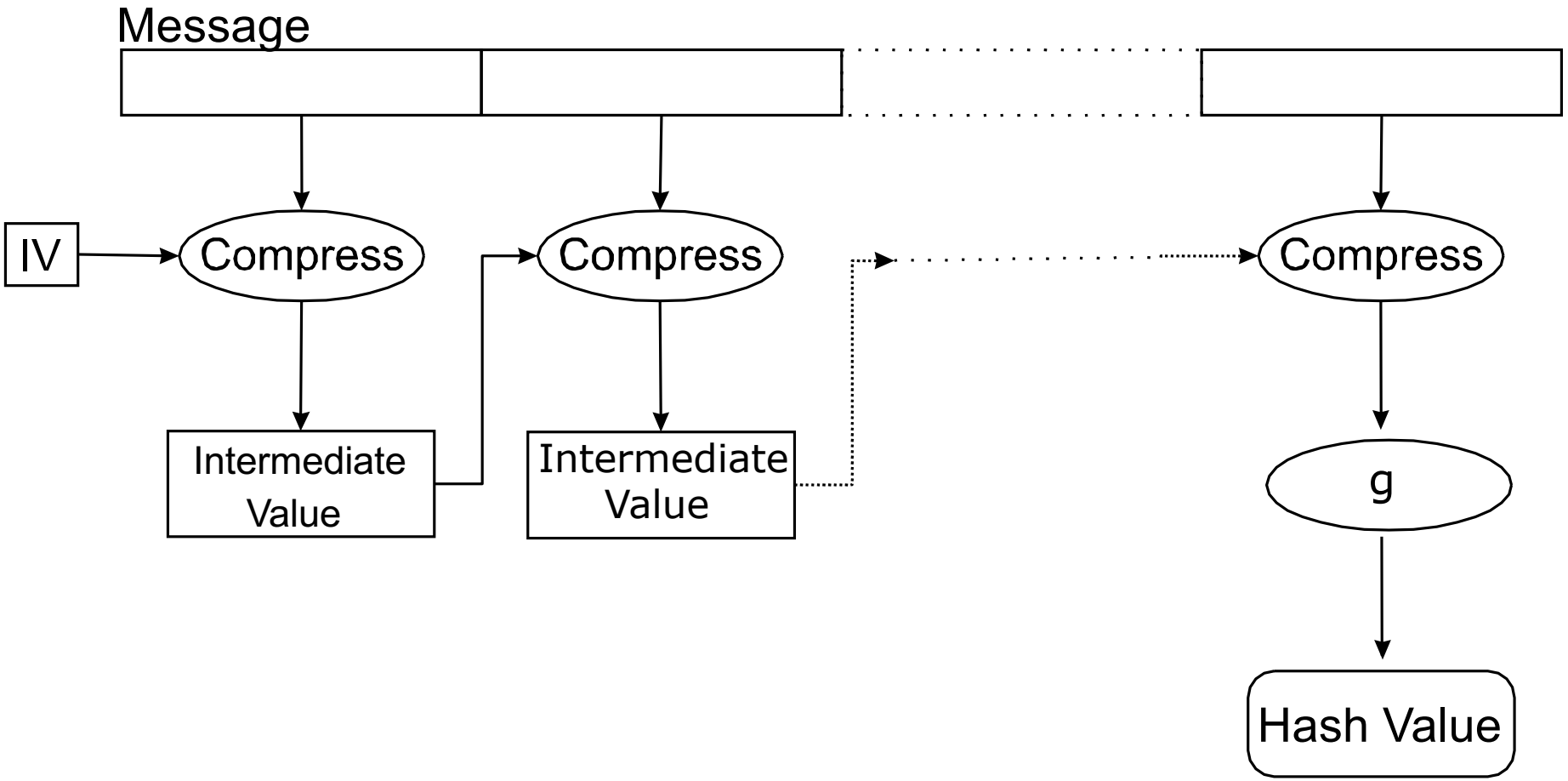
Construction

Pattern for the design of fast hash functions:

- The core of the hash function is a compression function f that works on fixed size input blocks.
- An input x of arbitrary length is broken up into blocks $x_1,\ldots, x_m$ of the given block size; the last block has to be padded.
- Compute the hash of x by repeatedly applying the compression function: with a (fixed) initial value h_0, compute $h_i = f(x_i||h_{i-1})$ for $i=1,\ldots, m$ and take h_m as the hash value of x.

The symbol "||" denotes concatenation.

Hash Functions: A Typical Architecture



Hash function constructions

Loads of (50+) broken schemes:

- Rabin, Jueneman, X.509 Annex D, FFT-hash I and II, N-hash, Snefru, MD2, …

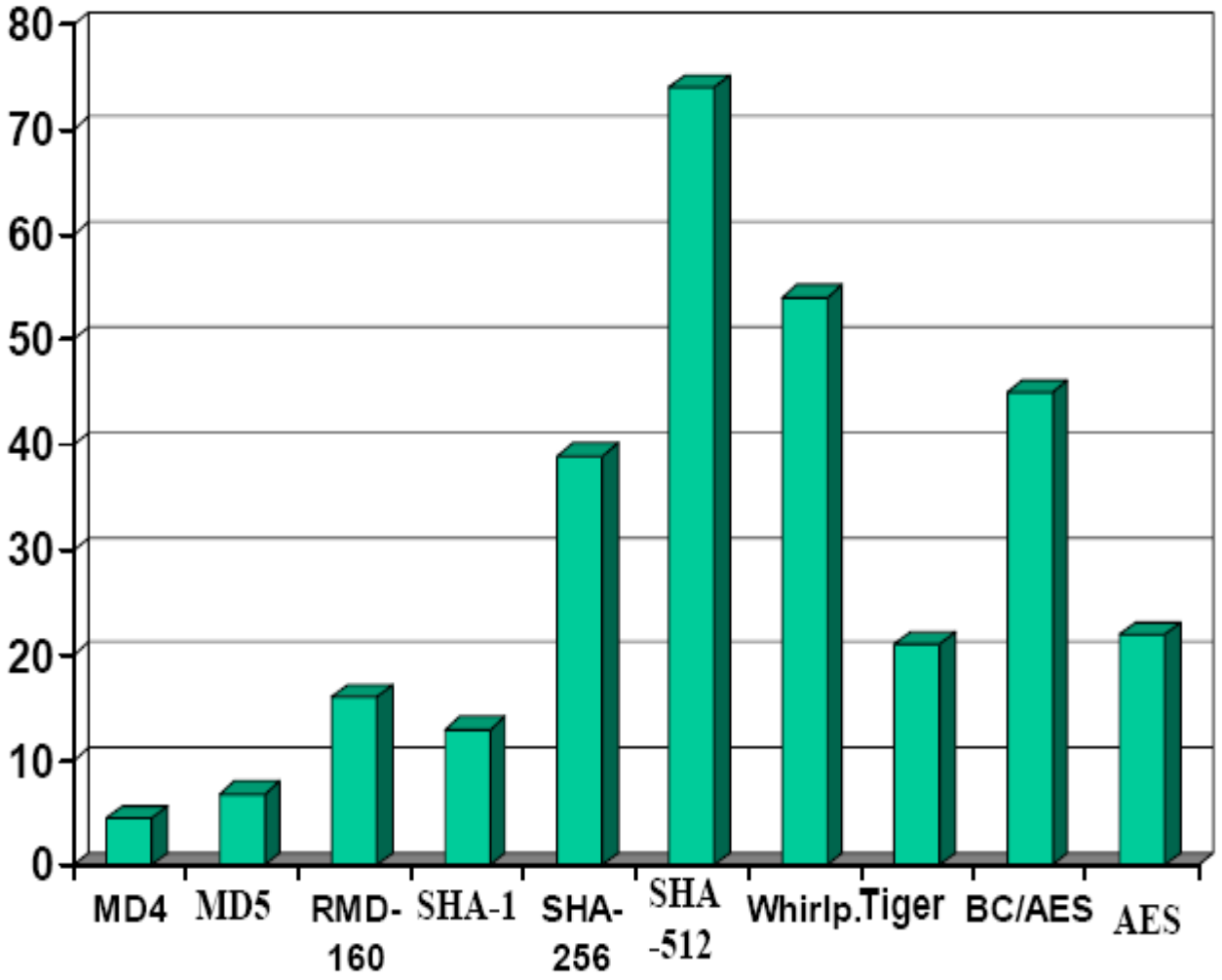
Fast schemes for 32-bit machines:

- popular early designs (broken): MD4 and MD5
- US government (NIST): SHA-*
- Europe: RIPEMD-160

Current generation: SHA-512

- Whirlpool, …
- SHA-3 (Keccak)

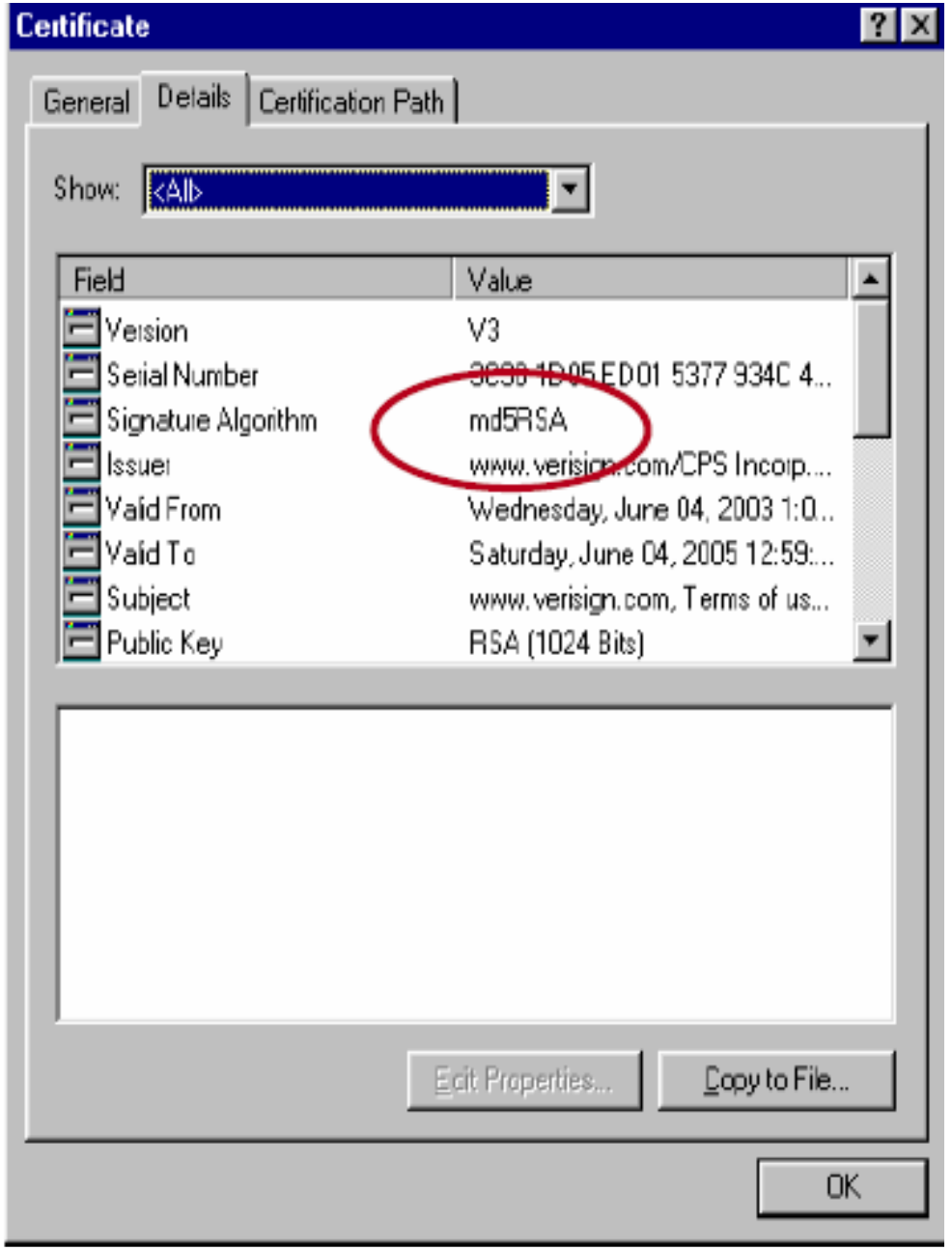
Performance of hash functions



Hash Functions – Real Life...

MD5

- Advice (RIPE since 1992, RSA since 1996): stop using MD5
- Largely ignored by industry (click on a cert...)
- Collisions for MD5 are within range of a brute force attack anyway (2^{64})
- [Wang+'04] collision in 15 minutes



https://schaerli.org/weblog/7-filen/ (Nov. 2026)

Pascal Schärli ◐

"Zero-knowledge encryption"

- MD5(MD4(MD2(password)
- Encrypt like it's 1854
- Stranger keys

Crypto doesn't matter*

*if you cannot trust the code



Filen's password onion made me cry

2026-11-03

Filen is a young German end-to-end encrypted cloud storage provider. Five years in, they are on the third version of their crypto protocol. The oldest accounts still authenticate with long-deprecated hash functions like MD2 and MD4, with attacks dating back to the last millennium. A downgrade attack could trick even recent accounts into using them, which for weak passwords could leak all files to the service provider.

This post walks through some of the cryptographic weaknesses I found and questions if end-to-end encryption is worth it when delivered on a webpage. I responsibly disclosed every finding to Filen and gave them a draft of this post to review before publication. You can find the unabridged list of findings here.

History of Hash Functions

- MD4 introduced (3 round algorithm) [Rivst, 1990]
- Serious flaws in MD4 (attack on last two rounds) [Boer ’91]
- SHA replaced by SHA-1 [NSA ’94]
- Collisions for MD4, problem in ext.-MD4 [Dobbertin ’96]
- More problems of MD5 and RIPEMD [Dobbertin ’96]
- Collisions for Haval [Biryukov, Van Rompay, Preneel ’02]
- Collisions for SHA-0 [Joux ’04]
- Collisions for MD4 (by hand), MD5, and RIPEMD [Wang, Feng, Lai, Yu ’04]
- Attack on 53 out of 80 rounds of SHA-1 [Oswald-Rijmen’04 and Biham-Chen] ’04]
- 2^{39} attack on SHA-0 [Wang, Yu, Yin ’05]
- 2^{69} attack on SHA-1 [Wang, Yin, Yu ’05]
- 2^{63} attack on SHA-1 [Wang, Yao, Yao ’05]

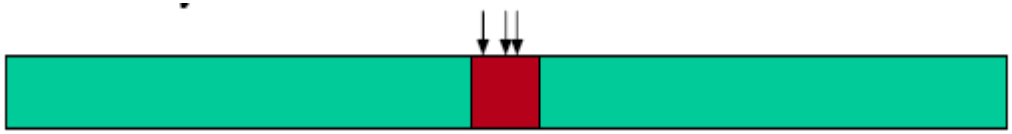
History of MD5 hash

- Published 1991 by Rivest
- Pseudo-collision [Boer et al. ’93] (limited practical value)
- Collision on the compression function [Dobbertin ’96]
  - Conclusion: in the future MD5 should no longer be implemented...where a collision-resistant hash function is required.
- Identical-prefix collisions for MD5 [Wang, Feng, Lai, Yu ’04]
  - Two 128-byte messages with same hash
  - MD5(A|M|B) = MD5(A|M‘|B) (i.e. two messages differn in 128 consecutive „random“ bytes)
- Choosen prefix collision for MD5 [Stephens et al, ’06]
  - Two arbitrary (same-length) files
  - Append 716 „random“ bytes to make them collide
- (Theoretical) preimage attack (2009): $O(2^{123.4})$

Impact of collisions

Collisions for MD5, SHA-0, SHA-1

- two messages differ in a few bits in 1 to 3 512-bit input blocks
- limited control over message bits in these blocks, but arbitrary choice of bits before and after them



What is achievable?

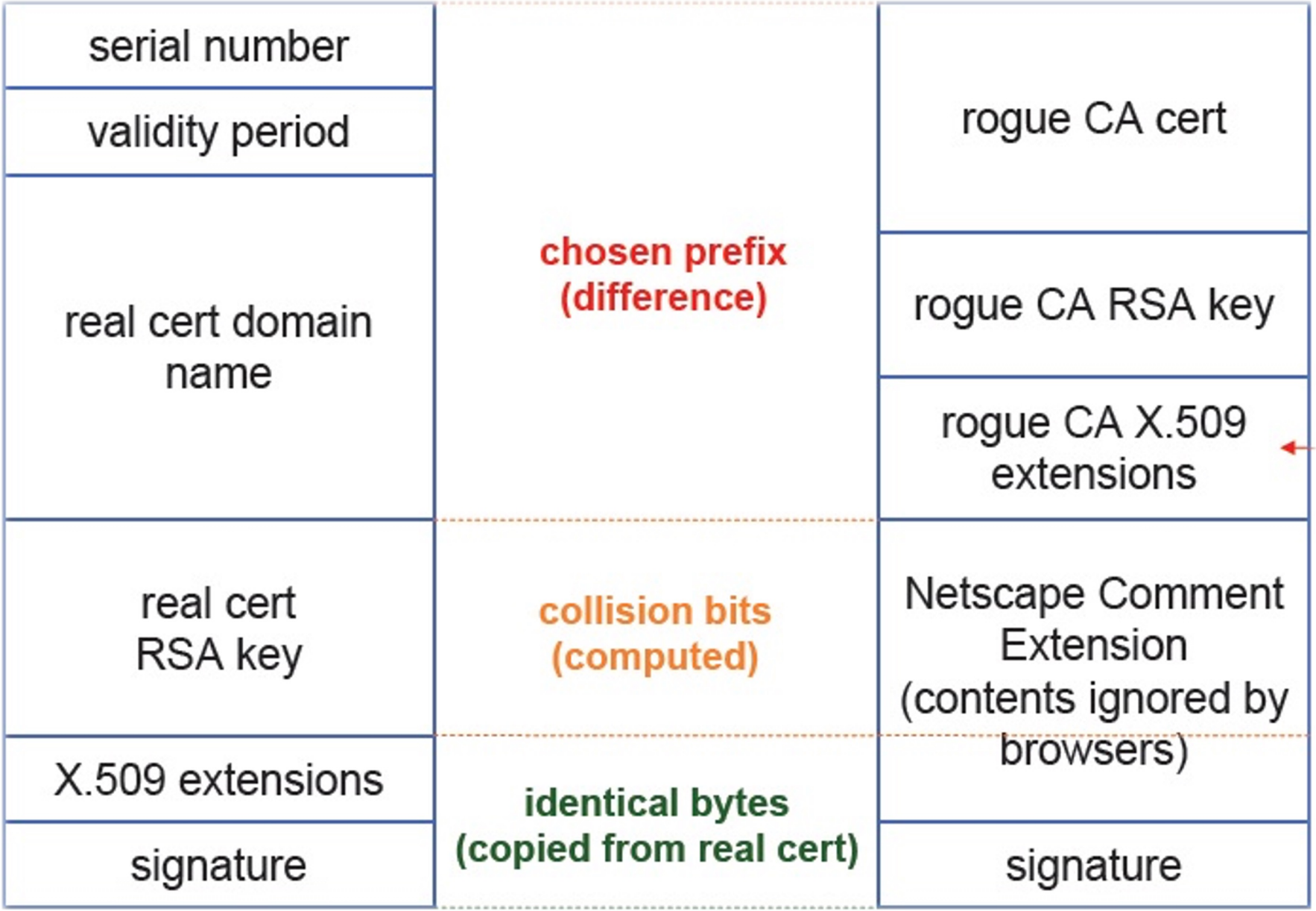
- 2 colliding executables
- 2 colliding postscript documents [Lucks, Daum ’05]
- 2 colliding RSA public keys – thus with colliding X.509 certificates [Lenstra, Wang, de Weger ’04]
- 2 arbitrary colliding files (no constraints) for 100K$

Impact of collisions

What is achievable? (2)

- **Rouge CA certificate**
  - Challenge 1: finding a (useful) collision
    - 1-2 days on a cluster of 200 PlayStation 3
    - Computing power equivalent to $20.000 on Amazon EC2
  - Challenge 2: get a CA signature
    - make some CA sign a (the right) certificate for you

Rouge CA certificate



NOVEMBER 4, 2015 10:00 AM / by Microsoft Edge Team

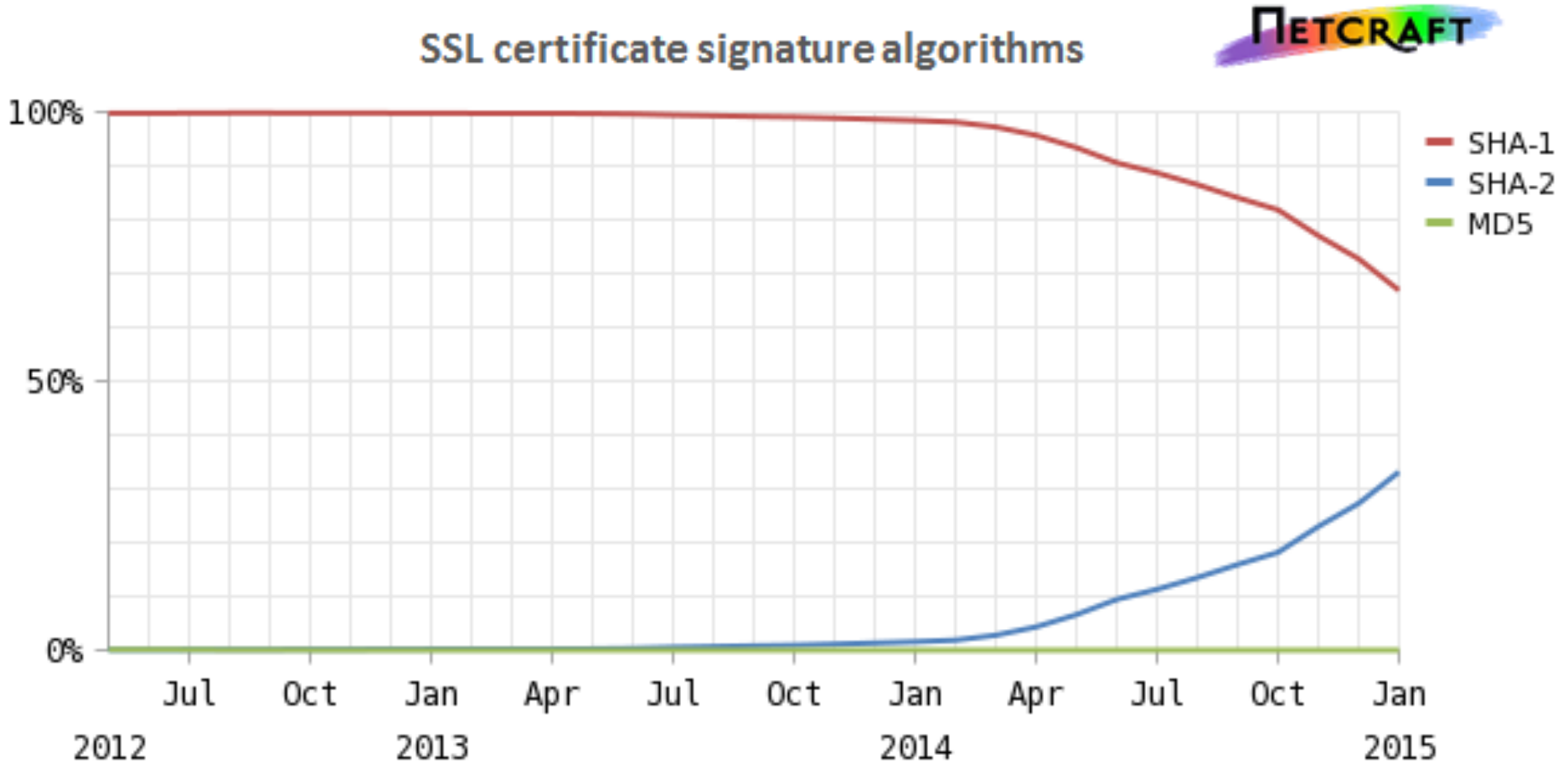
SHA-1 Deprecation Update

In a previous update on TechNet, we announced that Windows will block SHA-1 signed TLS certificates starting on January 1, 2017. In light of recent advances in attacks on the SHA-1 algorithm, we are now considering an accelerated timeline to deprecate SHA-1 signed TLS certificates as early as June 2016.

Mozilla recently announced a similar intent on the Mozilla Security Blog. We will continue to coordinate with other browser vendors to evaluate the impact of this timeline based on telemetry and current projections for feasibility of SHA-1 collisions.

For more details on our schedule, please see Windows Enforcement of Authenticode Code Signing and Timestamping on Technet, or reach out to us on Twitter or in the comments below.

– Kyle Pflug, Program Manager, Microsoft Edge

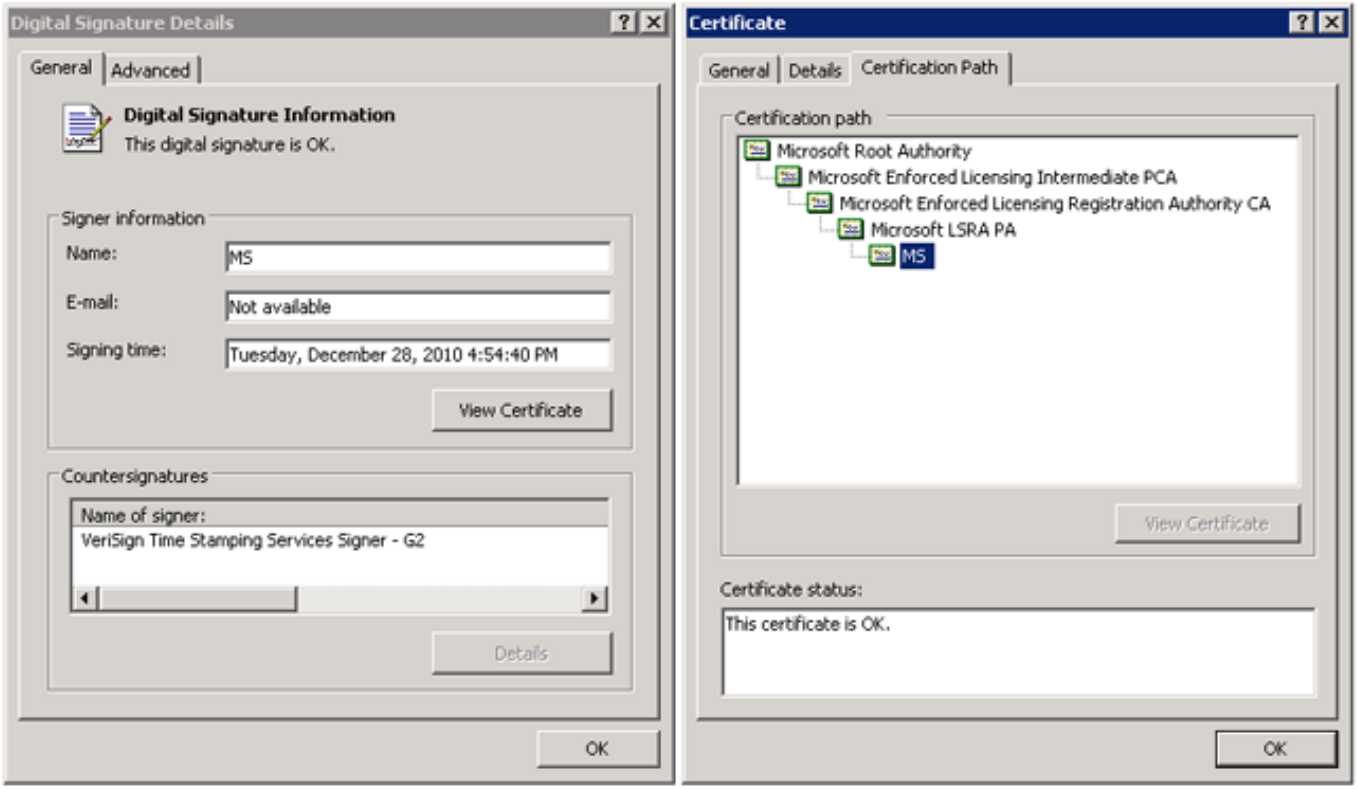

SSL certificate signature algorithms
NETCRAFT
100%
50%
0%
SHA-1
SHA-2
MD5
Jul
Oct
Jan
Apr
Jul
Oct
Jan
Apr
Jul
Oct
Jan
2012
2013
2014
2015

Flame malware hijacks Windows Update to spread from PC to PC

It's hard to patch a machine when the update mechanism is compromised.

by **Dan Goodin** - Jun 5, 2012 12:30am CEST

38



The certification path of the certificate used to sign WuSetupV.exe, which masquerades as a legitimate Windows Update from Microsoft.

F-Secure

The Flame espionage malware targeting Iranian computers contains code that can completely hijack the Windows update mechanism that Microsoft uses to distribute security patches to hundreds of millions of its users, security researchers said Monday.

FLAME

Revealed: Stuxnet "beta's" devious alternate attack on Iran nuke program

Microsoft speaks out on Flame malware certificate forgery

07 JUN 2012 12

Cryptography, Malware, Microsoft



In the Flame case, the attackers took a legitimate Microsoft certificate using MD5 for its hash and RSA-2048 for its public-key encryption. They then generated a similar-but-different certificate with the same MD5 hash. This means that the RSA-2048 signature from Microsoft's genuine certificate could be grafted into their forged certificate to make it appear valid.

Message Authentication Codes

Cryptographic Hash:

- Do not depend on any secret key
- I.e., anyone key calculate the hash
- For integrity checks, hash values have to be protected!

Message Authentication Code (MAC)

- Similar to cryptographic hash
- But: dependent on a secret key
- Knowledge of key necessary for calculating (or verifying) the message authentication code
- *keyed hash function* for *data origin authentication*

Message authentication codes

Simple constructions:

“secret prefix”: calculate $h(k \| m)$

“secret suffix”: calculate $h(m \| k)$

“secret envelope”: calculate $h(k_1 \| m \| k_2)$

Security analysis:

Preneel & van Oorschot: „MDx-MAC and Building Fast MACs from Hash Functions“ (Crypto‘95)

http://www.ietf.org/rfc/rfc2104.txt (11 pages)

Network Working Group
Request for Comments: 2104
Category: Informational

H. Krawczyk
IBM
M. Bellare
UCSD
R. Canetti
IBM
February 1997

HMAC: Keyed-Hashing for Message Authentication

Status of This Memo

This memo provides information for the Internet community. This memo does not specify an Internet standard of any kind. Distribution of this memo is unlimited.

Abstract

This document describes HMAC, a mechanism for message authentication using cryptographic hash functions. HMAC can be used with any iterative cryptographic hash function, e.g., MD5, SHA-1, in combination with a secret shared key. The cryptographic strength of HMAC depends on the properties of the underlying hash function.

Message authentication codes

HMAC construction: take a hash function *h*, for a key *k* and a document *x*, compute

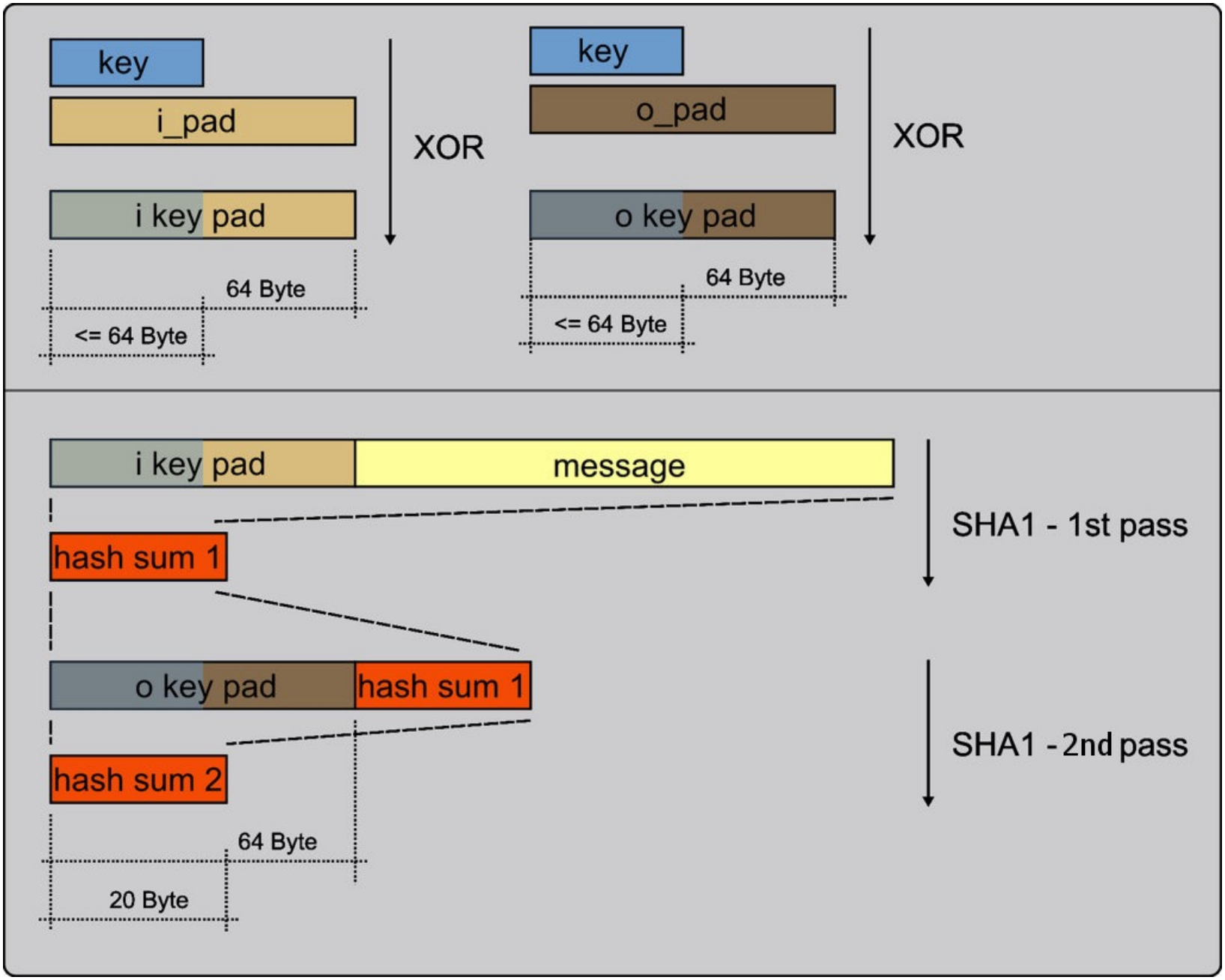
$$HMAC(x) = h(k \oplus o_pad \,||\, h(k \oplus i_pad \,||\, x))$$

(*i_pad*, *o_pad* are padding fields, "||" means concatentation)

Further Reading: M. Bellare, R. Canetti, and H. Krawczyk: Message authentication using hash functions: The HMAC construction, RSA Laboratories' CryptoBytes Vol. 2, No. 1, Spring 1996.

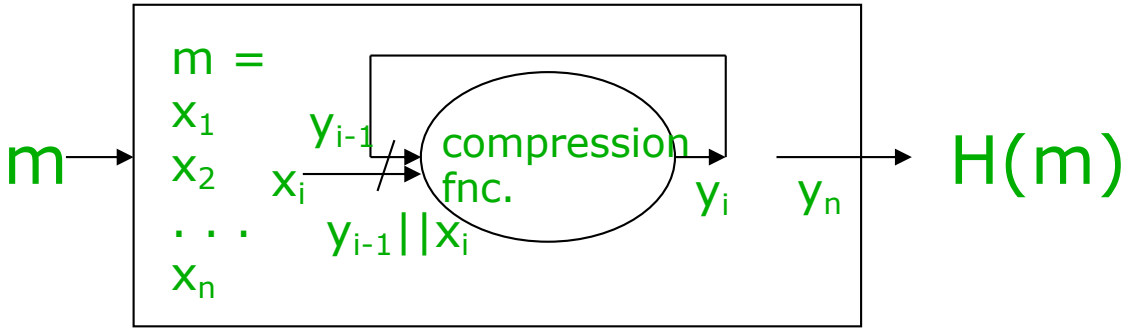
RFC2104

Message authentication codes



Internals of a Hash Function

- Fixed-size “compression function”.
- Iteration mixes an input block with the prev. output.



Internals:

- Lots of (fast) operations (rotations, $\oplus$, $\wedge$, $\vee$, +, ...)

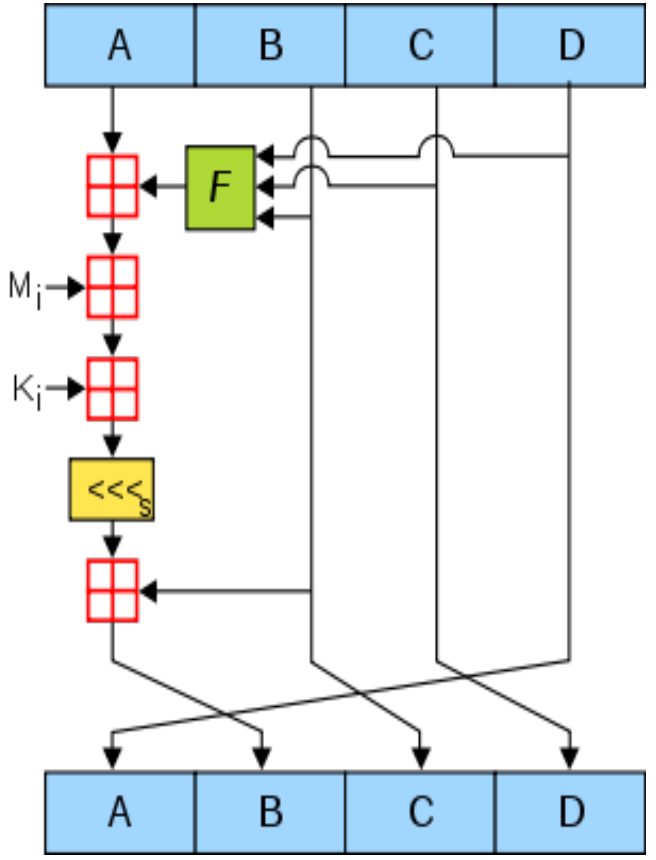
No good theory for general design (so far)

MD5

Rivest, 1991

2004: First collusion attacks discovered.

2008: Practical collision attack, generating SSL certificates with the same MD5 hash.



M_i is a 32-bit block of the message input, and K_i a 32-bit constant, different for each operation. s varies for each operation. ⊞ denotes addition modulo 2^{32}.

SHA-1

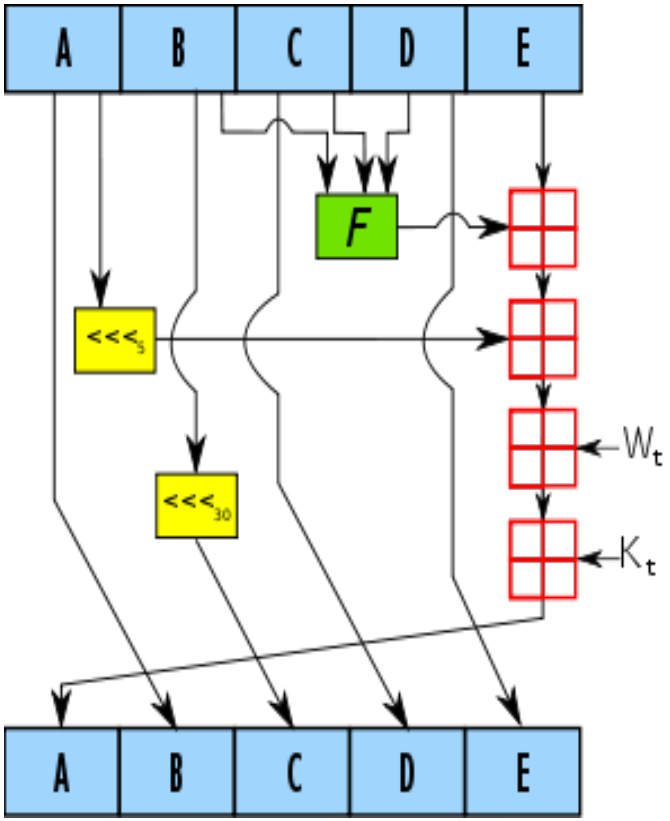
Designed by NSA; based on Rivest's MD4 & MD5 designs

SHA 1993; SHA-1 1995

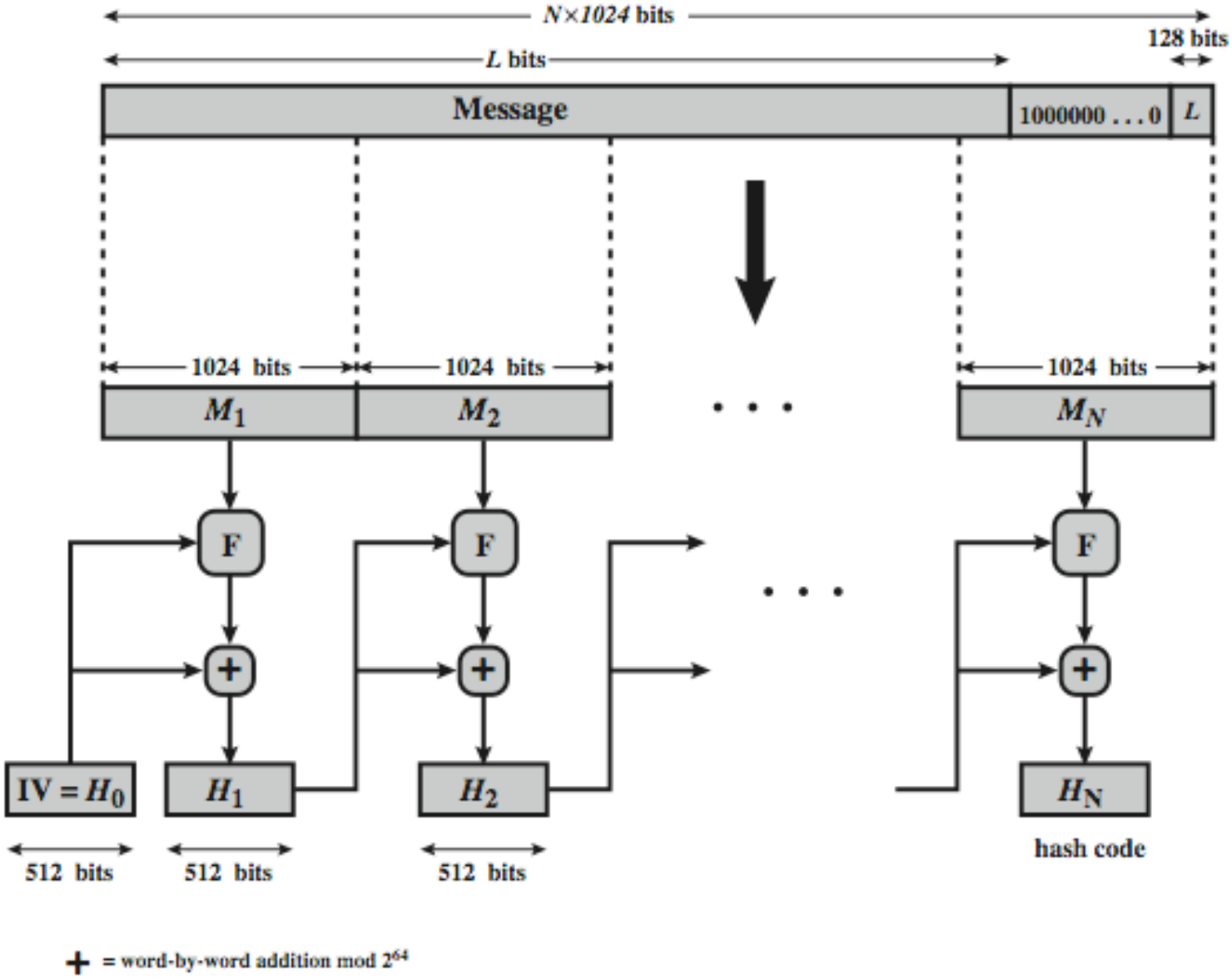
160-bit output size

2005: first flaws discovered.

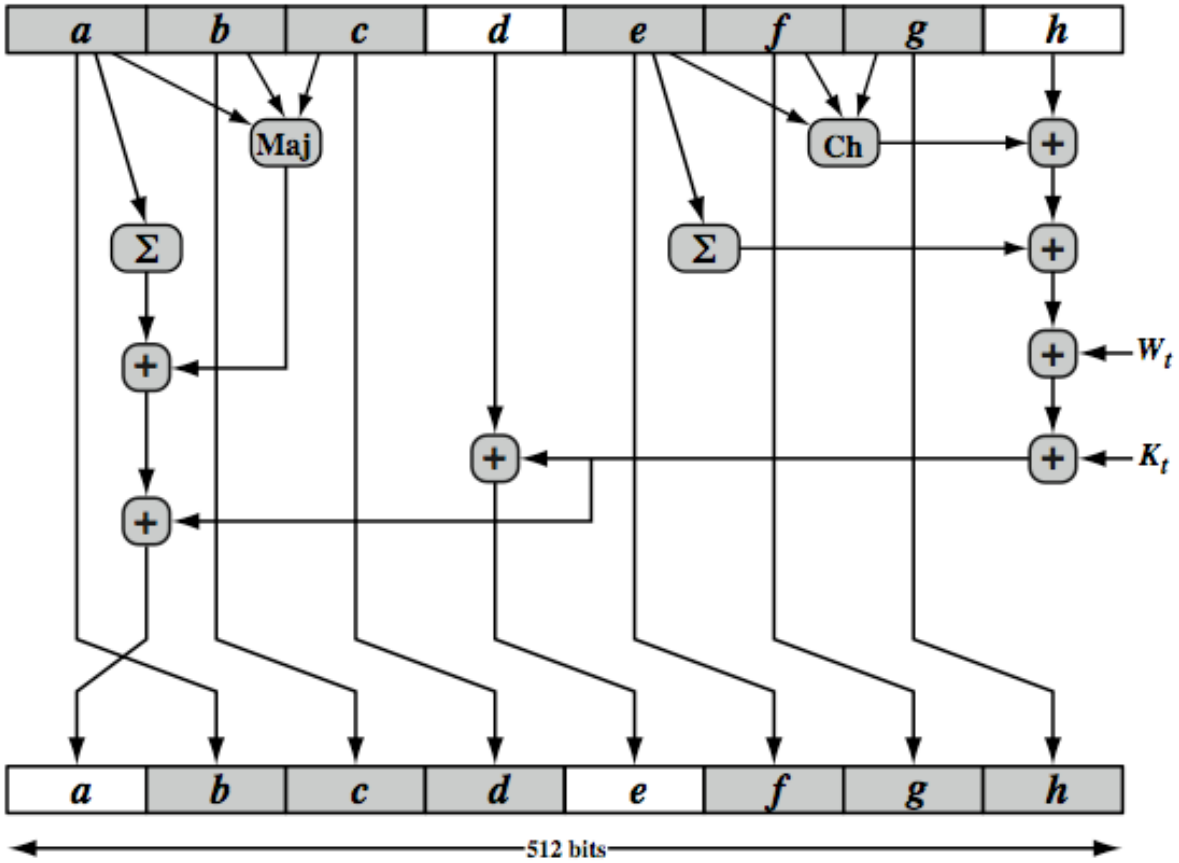
SHA-2: 256- and 512-bit extension



SHA-512 Overview

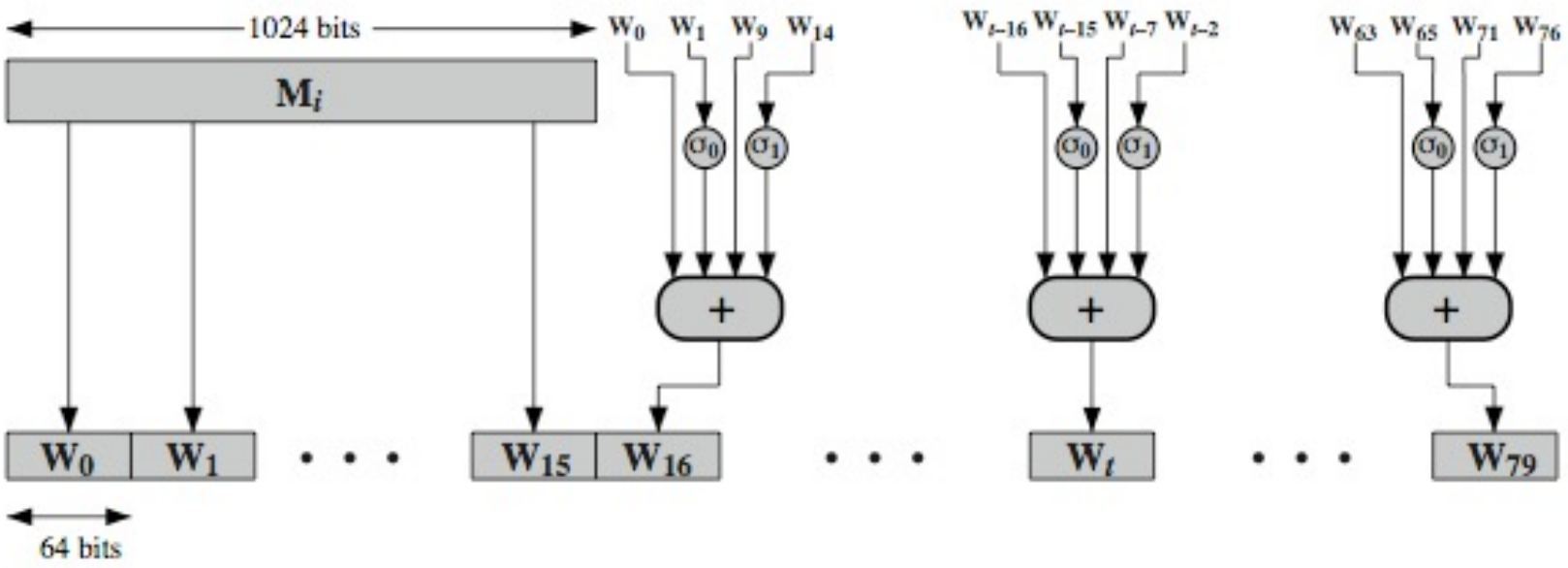


SHA-512 Round Function (F)



Ch(e,f,g) = (e AND f) XOR (NOT e AND g)
Maj(a,b,c) = (a AND b) XOR (a AND c) XOR (b AND c)
∑(a) = ROTR(a,28) XOR ROTR(a,34) XOR ROTR(a,39)
∑(e) = ROTR(e,14) XOR ROTR(e,18) XOR ROTR(e,41)

SHA-512 Round Function



∂0(x) = ROTR(x,1) XOR ROTR(x,8) XOR SHR(x,7)
∂1(x) = ROTR(x,19) XOR ROTR(x,61) XOR SHR(x,6)

Encryption

Encryption algorithms (ciphers) protect the confidentiality of data

Some encryption algorithms can also be used for integrity checks

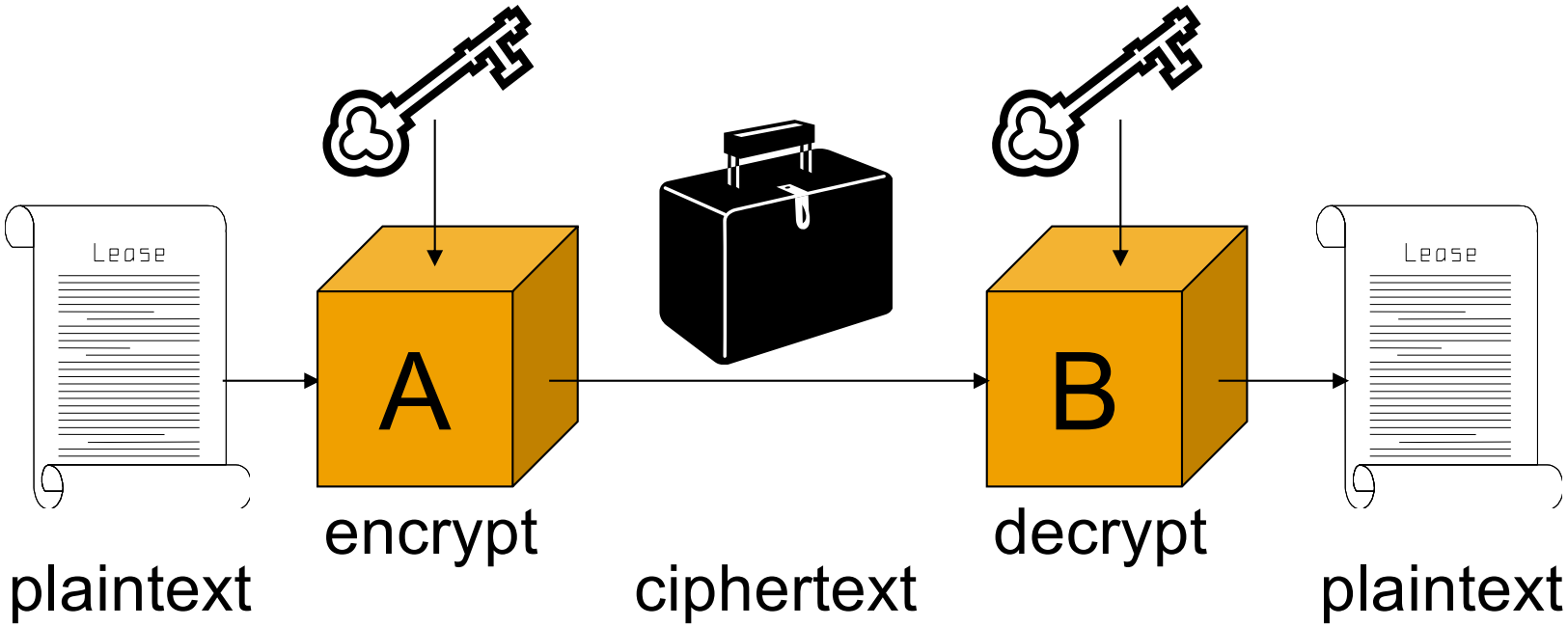
A plaintext (clear text) *x* is converted into a ciphertext under the control of a key *K*

- we write *eK(x)*

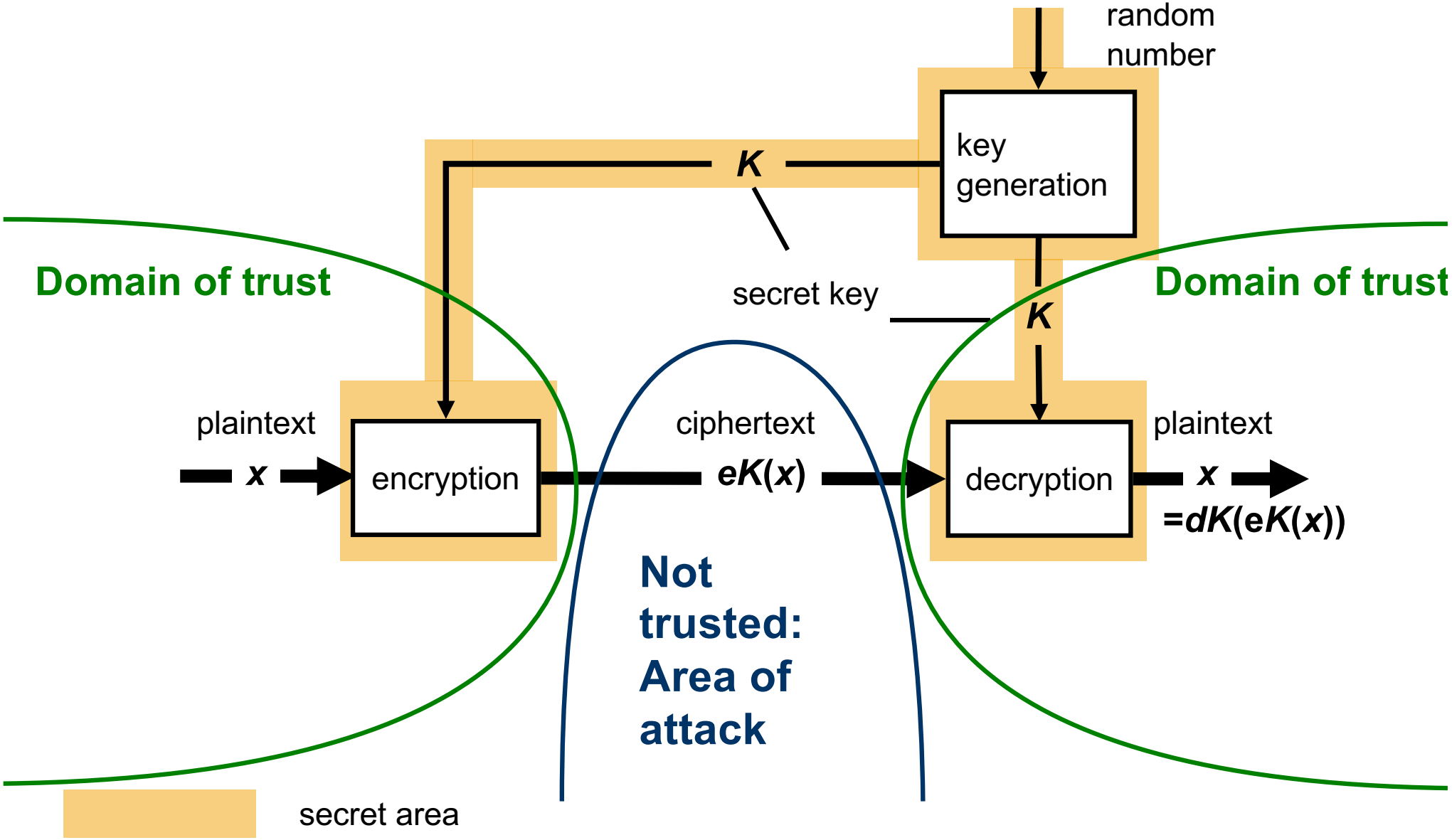
Decryption with an appropriate key computes the plaintext from the ciphertext

- we write *dK(x)*

Symmetric key encryption



Symmetric Encryption System



Cryptanalysis

Cryptanalysis: science of recovering the plaintext from ciphertext without the key.

Always assume attackers know the algorithms used

- **Algorithms can be published to facilitate the evaluation of their security**
- **Security should depend on secrecy of the key, not the algorithm**

Contrast with security by obscurity.

- **Analogy: Hide a letter under your mattress versus lock it in a safe, whose design has been published and whose locking mechanism has withstood attacks from the world's best safecrackers.**